

FACTORY DESIGN PATTERN - COMPLETE NOTES

(Simple Language, Step by Step)

STEP 1: THE PROBLEM - COMPOSITION WITH CONCRETE CLASS

Suppose we have a class called ModifyLogic. This class wants to convert a file into PDF format. So we do composition (one class using object of another class) like this:

```
class PDFMessageConverter {
    public String convertTo(String fileName) {
        return "<PDF>" + fileName + "</PDF>";
    }
}

class ModifyLogic {
    PDFMessageConverter converter = new PDFMessageConverter(); // CONCRETE CLASS OBJECT
    public String modify(String fileName) {    return converter.convertTo(fileName); }
}
```

PROBLEM:

Here ModifyLogic is tightly bound (tightly coupled) to PDFMessageConverter.

Tomorrow if user wants WORD conversion also, then we need to:

- Create a new object of WORDMessageConverter
- Change the code of ModifyLogic class again and again
- Every time a new type comes (Excel, HTML, etc), we must edit ModifyLogic class

This is bad because:

1. We are repeatedly modifying the same class - violates "Open Closed Principle"
2. (class should be open for extension, closed for modification)
3. ModifyLogic class becomes dependent on every concrete class
4. Code becomes hard to maintain and not flexible

STEP 2: SOLUTION - USE INTERFACE (STRATEGY DESIGN PATTERN)

Instead of using concrete class directly, we create a common INTERFACE. Every converter class (PDF, WORD) will implement this interface.

```
interface Logic {
    public String convertTo(String fileName);
}

class PDFMessageConverter implements Logic {
    public String convertTo(String fileName) {
        return "<PDF>" + fileName + "</PDF>";
    }
}

class WORDMessageConverter implements Logic {
    public String convertTo(String fileName) {
        return "<html>" + fileName + "</html>";
    }
}

class ModifyLogic {
    Logic converter; // NOW WE USE INTERFACE TYPE, NOT CONCRETE CLASS
    public String modify(String fileName, Logic converter) {
        this.converter = converter;
        return converter.convertTo(fileName);
    }
}
```

WHY THIS IS BETTER (This is called STRATEGY DESIGN PATTERN):

- ModifyLogic class no longer depends on a specific concrete class
- It depends only on the interface "Logic"
- Whichever object (PDF or WORD) we pass at runtime, that strategy will be used
- We can add 10 more converters in future without touching ModifyLogic class
- This is called "Program to an Interface, not an Implementation"

PROBLEM STILL LEFT:

But now the user of ModifyLogic class has to create the object himself:

Logic converter = new PDFMessageConverter(); // user still creating object

We want the OBJECT CREATION part to also be separated, so that user does not worry "which class to create". This is where FACTORY PATTERN comes in.

STEP 3: WHAT IS FACTORY DESIGN PATTERN?

Factory Design Pattern is a Creational Design Pattern.

Simple Meaning:

"Factory class is responsible for creating the object of some other class, so that the calling class does not need to create the object by itself."

Just like in real life - we don't manufacture a car ourselves, we go to a CAR FACTORY and tell our requirement, and the factory gives us the ready car.

Similarly here:

- We tell the Factory "I want PDF" or "I want WORD"
- Factory creates and returns the correct object
- We (calling class) don't need to know "new PDFMessageConverter()" or "new WORDMessageConverter()"

BENEFITS OF FACTORY PATTERN:

1. Object creation logic is in one single place (centralized)
2. Calling class becomes simple, clean, and loosely coupled
3. Easy to add new types in future, only Factory class needs change
4. Hides the complex object creation logic from the user

STEP 4: HOW FACTORY CLASS WORKS HERE

```
class MessageFactory {
    public static Logic factoryMethod(String type) {
        Logic data = null;
        if (type.equalsIgnoreCase("pdf")) {
            data = new PDFMessageConverter();
        }
        else if (type.equalsIgnoreCase("word")) {
            data = new WORDMessageConverter();
        }
        else {
            data = null;
        }
        return data;
    }
}
```

Here, based on the "type" string passed (pdf/word), Factory decides which object to create and return.

The calling class (ModifyLogic) does not know or care how the object is created.

NOTE: In the original code, Class.forName() and newInstance()

(Reflection) were used to create object dynamically.

That is an advanced way of doing the same thing - creating object using class name as String.

For learning purpose, simple "new ClassName()" is enough and easy to understand.

STEP 5: FINAL FLOW (HOW EVERYTHING CONNECTS)

1. Logic Interface --> defines the contract (convertTo method)
2. PDFMessageConverter, WORDMessageConverter --> implement Logic (Strategy classes)
3. MessageFactory --> creates correct object based on type (Factory)
4. ModifyLogic --> uses Logic interface, gets object from Factory, calls method (Client)
5. MyApp --> main class, takes user input and starts the process

FULL FINAL CODE

```
package com.tak.day07;
import java.util.Scanner;
```

```
interface Logic {
    public String convertTo(String fileName);
}
```

```
class PDFMessageConverter implements Logic {
    public String convertTo(String fileName) {
        return "<PDF>" + fileName + "</PDF>";
    }
}
```

```
class WORDMessageConverter implements Logic {
    public String convertTo(String fileName) {
        return "<html>" + fileName + "</html>";
    }
}
```

```
class MessageFactory {
    public static Logic factoryMethod(String type) {
        Logic data = null;
        if (type.equalsIgnoreCase("pdf")) {
            data = new PDFMessageConverter();
        }
        else if (type.equalsIgnoreCase("word")) {
            data = new WORDMessageConverter();
        }
        else {
            data = null;
        }
        return data;
    }
}
```

```
class ModifyLogic {
    Logic converter;
    public String modify(String fileName, String type) {
        converter = MessageFactory.factoryMethod(type);
        String data = converter.convertTo(fileName);
        return data;
    }
}
```

```
public class MyApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("\u001B[31m" + "\t\tWelcome\t\t" + "\u001B[0m");
        System.out.println("\u001B[32m" + "Upload your file ");
        String file = sc.nextLine();
        System.out.println("\u001B[33m" + "Enter your choice for conversion");
        System.out.println("\u001B[34m" + "Press Word for converting in word");
        System.out.println("\u001B[35m" + "Press PDF for converting in PDF" + "\u001B[0m");
        String type = sc.next();
        ModifyLogic logic = new ModifyLogic();
        String result = logic.modify(file, type);
        System.out.println("Wait for a while");
        try {
            Thread.sleep(3000);
        }
        catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(result);
    }
}
```